

COMMENT PROPOSER UNE ARCHITECTURE CONTENEURISÉE POUR RÉPONDRE AUX BESOINS DE SERVICES EN « SELF-DÉPLOIEMENT » ET D'INTÉGRATION CONTINUE ?

PRÉSENTÉ PAR DONATIEN ENEMAN



kubernetes



MESOS



MARATHON



git



docker



argo

VM OU CONTENEUR ?

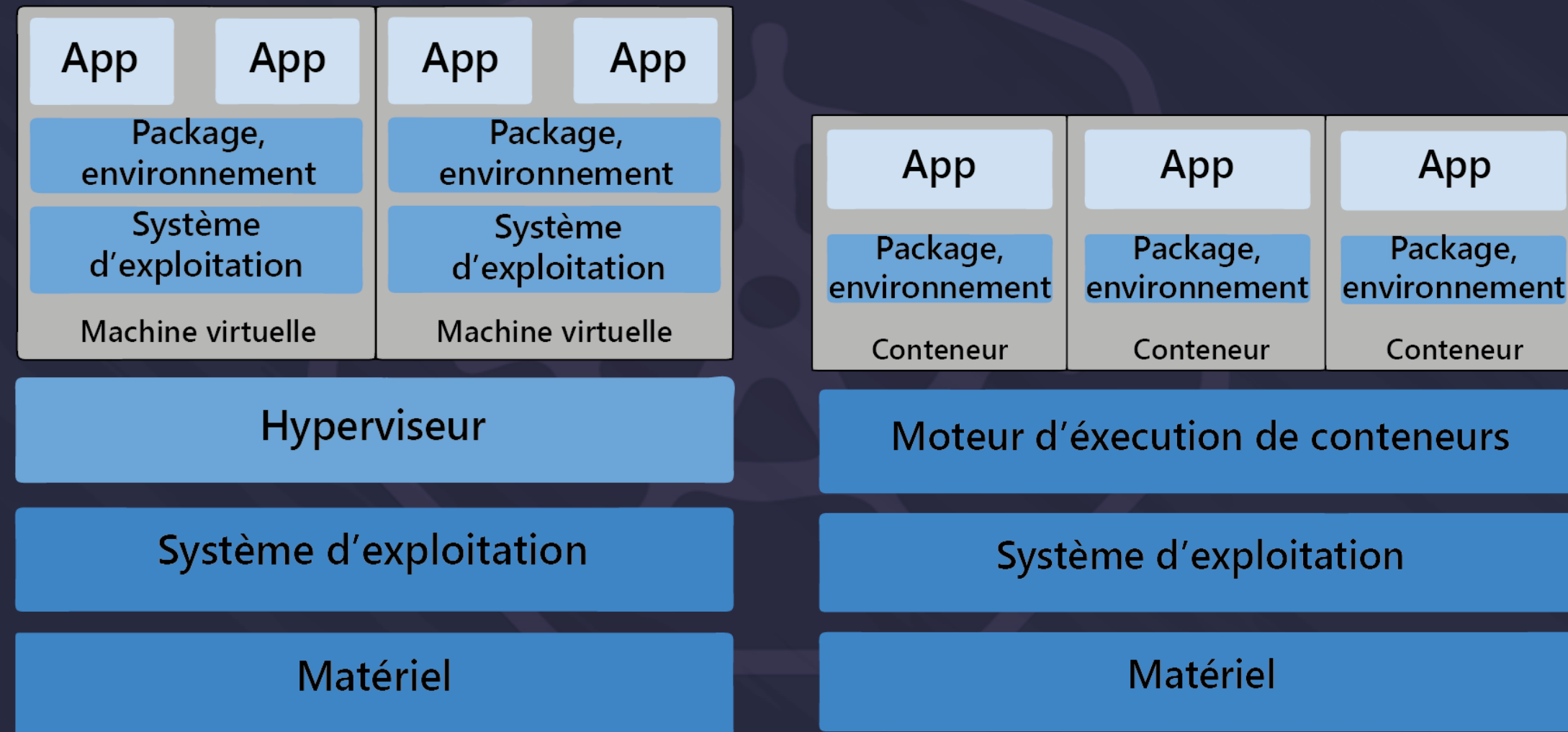


Schéma – Machine virtuelle

Schéma – Conteneur

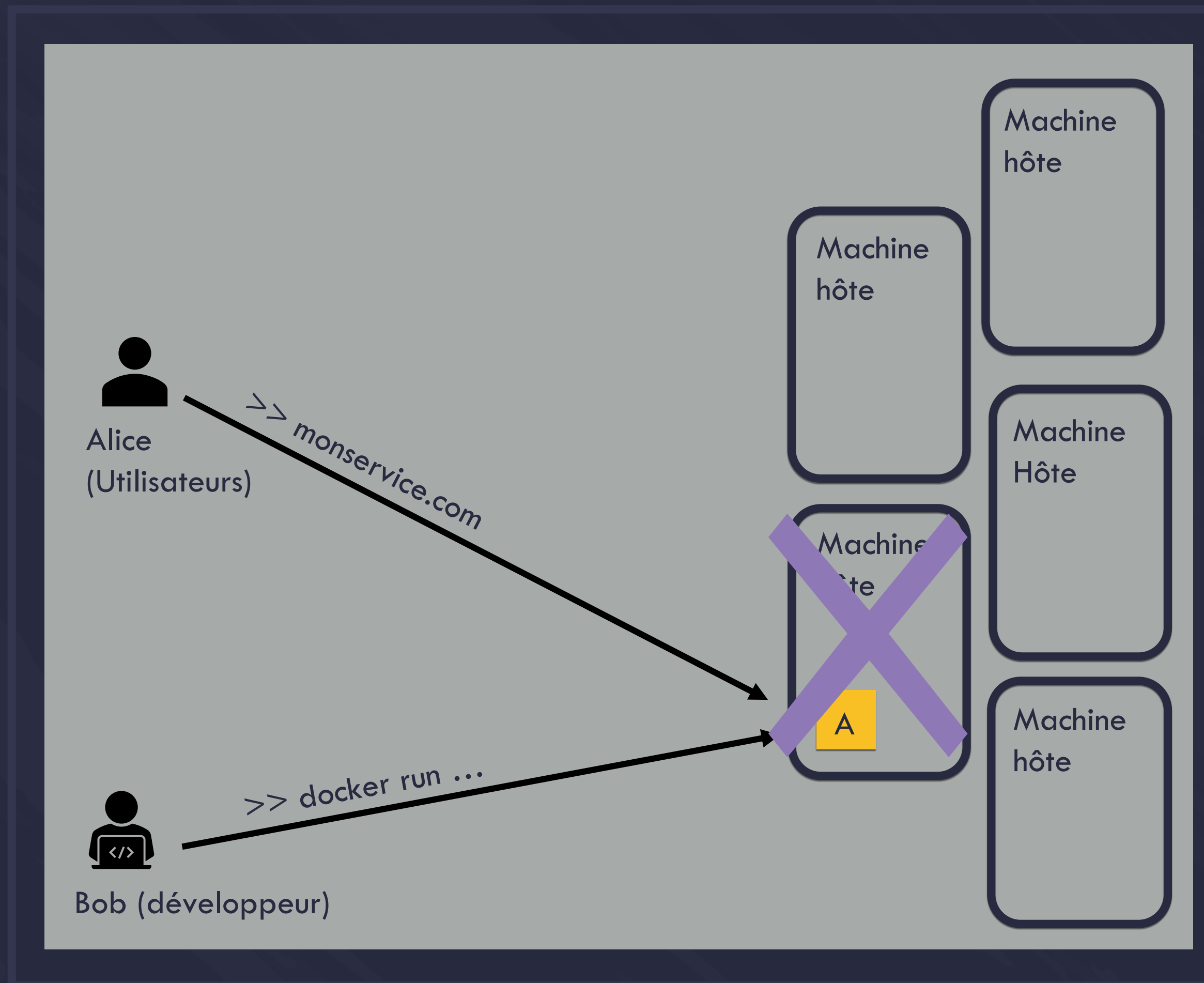
POURQUOI CHOISIR LES CONTENEURS ?

- ✓ Légèreté
- ✓ Reproductibilité et facilitée de déploiement.
- ✗ Une nouvelle approche de la sécurité
- ✗ Une nouvelle approche de la sauvegarde



On veut créer un environnement de tests pour les développeurs et de services à la demande. On choisit donc les conteneurs.

DÉPLOYER UN CONTENEUR



INCONVÉNIENT DE LA MÉTHODE

- ✗ Pénible pour le développeur (déploiement, monitoring, etc.)
- ✗ Comment gérer la perte d'une machine ?
- ✗ Comment gérer une montée en charge ?

➡ Méthode non applicable à grande échelle

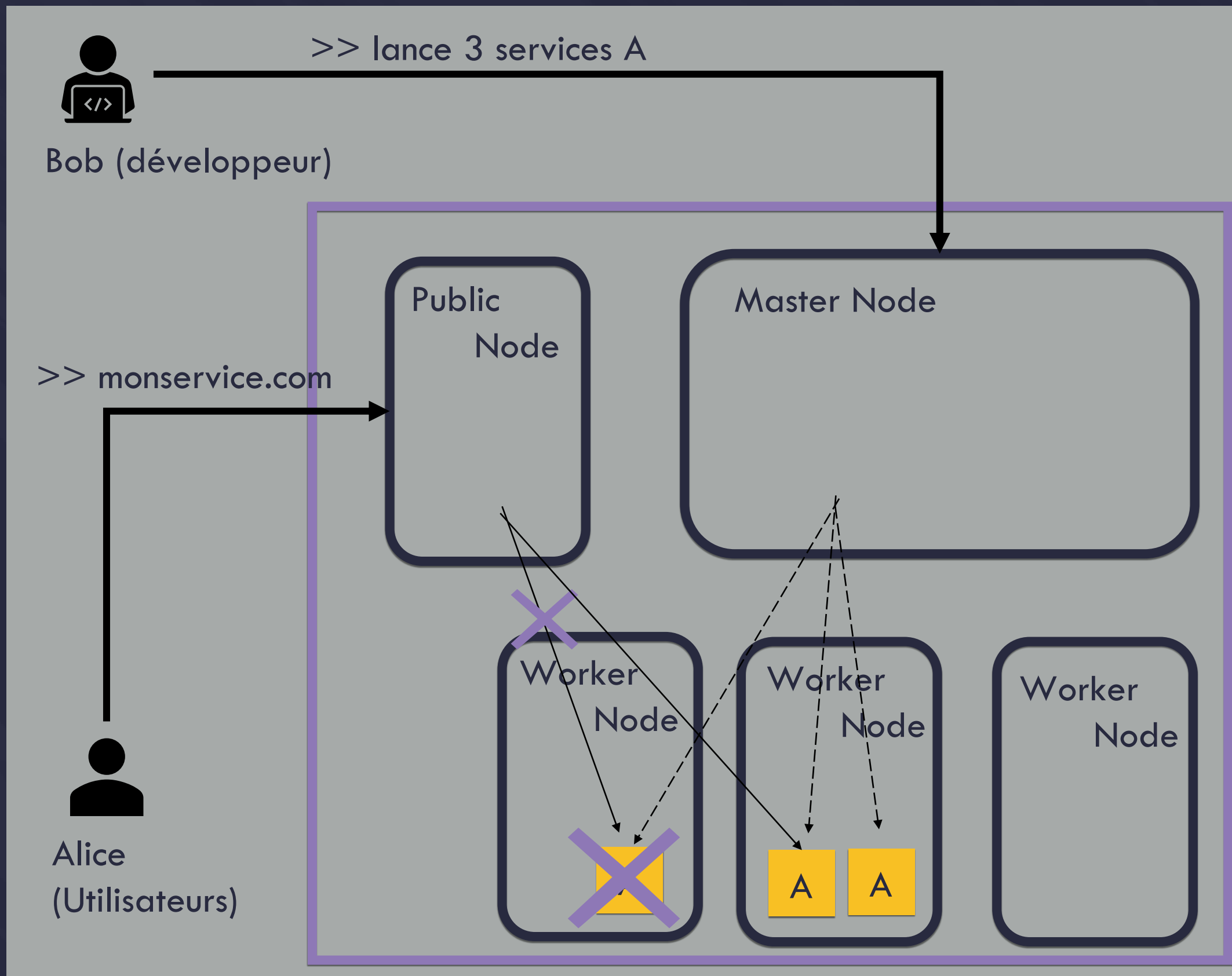
ORCHESTRATEUR DE CONTENEURS

3 TYPES DE MACHINES

- ✓ Nœuds de travail : machines non exposées disposant du plus de ressources et lançant les conteneurs
- ✓ Nœuds maître : machines qui gèrent l'état global du cluster
- ✓ Nœuds Public : machines permettant d'exposer les conteneurs

APPORT DE L'ORCHESTRATEUR

- ✓ Haute disponibilité
- ✓ Monitoring
- ✓ Service Discovery
- ✓ Gestion du trafic réseau
- ✓ Scalabilité horizontale et verticale
- ✓ Provisionning
- ✓ Placement des conteneurs



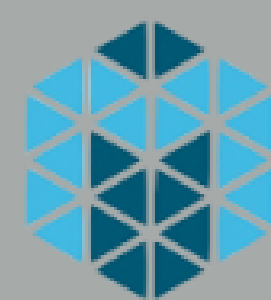
SOLUTIONS EXISTANTES



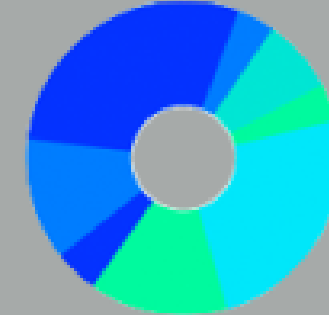
kubernetes

01

- ✓ OpenSource poussé par Google
- ✓ Le plus utilisé dans le monde



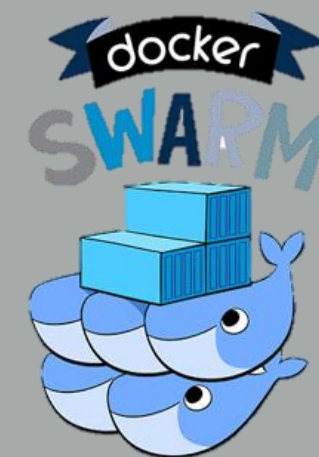
MESOS



MARATHON

02

- ✓ Le plus ancien des orchestrateurs
- ✓ Solution actuellement utilisée à l'innovation par le biais de DC/OS



03

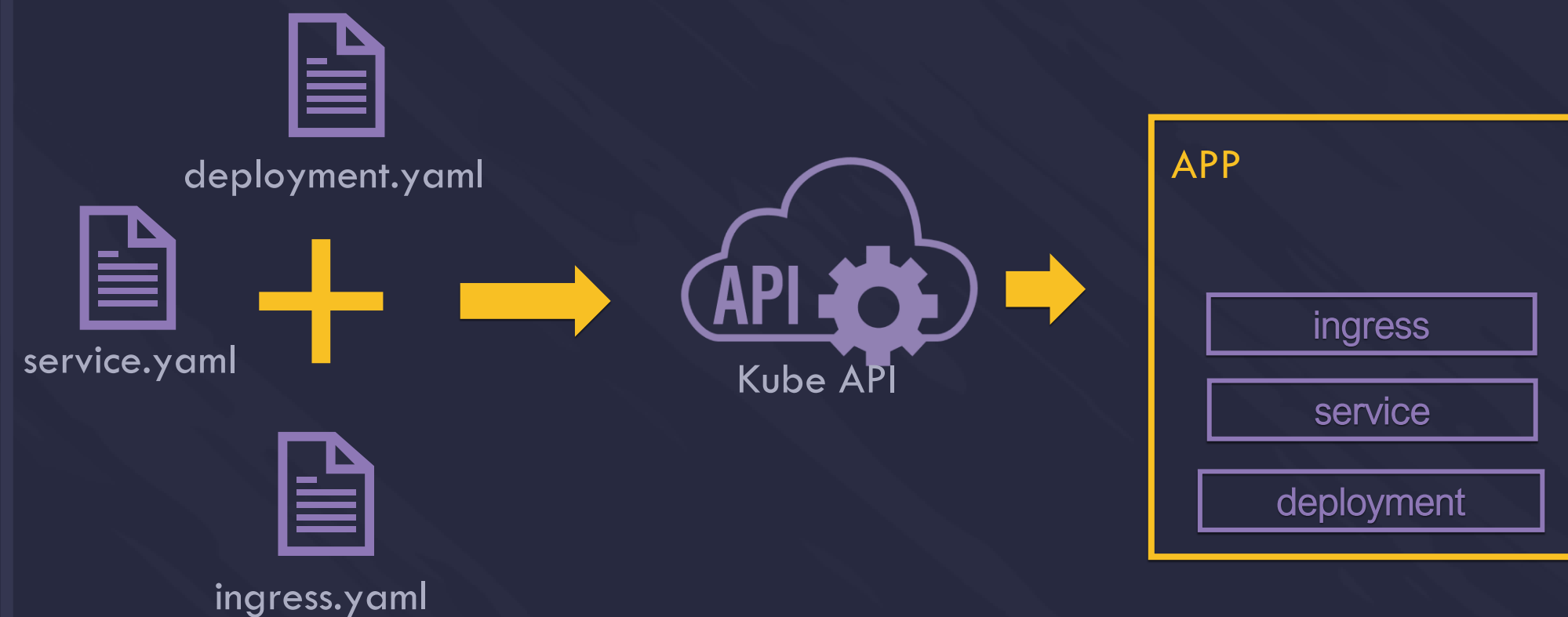
- ✓ Fourni par Docker
- ✓ Le plus récent



Pour ce stage, utilisation de Kubernetes par le biais de GCP et de Mesos/Marathon par le biais d'une Licence DC/OS entreprise.

DÉPLOYER UNE APPLICATION

UN FONCTIONNEMENT IDENTIQUE



KUBERNETES : MARATHON AVEC PLUS DE FONCTIONNALITÉS

- ✓ Séparation des ressources (namespace)
- ✓ Séparation des objets : deployment, service, ingress
- ✓ Des cas d'usage supplémentaires (téléchargement des données vs conteneur d'initialisation)

KUBERNETES : DE MEILLEURS OUTILS D'AIDE AU DÉVELOPPEMENT

- ✓ Des extensions (VScode) et des IDE (Lens)
- ✓ Des fonctionnalités supplémentaires (port-forward)
- ✓ Une CLI facile d'accès
- ✓ Des dashboards
- ✓ Plusieurs outils de templating (Helm, kustomize,...)

OBJECTIF 1 : METTRE EN PLACE DES SERVICES EN SELF-DÉPLOIEMENT



DES SERVICES A LA DEMANDE

RAPPEL OBJECTIF

- ➔ Proposer une plateforme permettant de lancer des services rapidement
- ➔ Minimum de configurations

MARATHON

- ✗ Aucune solution pré-existante
- ➔ Développement d'Onyxia

KUBERNETES

- ➔ KubeApps
 - ✓ Une UI permettant d'installer ou supprimer des charts, faire les montées de versions, etc.
 - ✓ Repose sur une API qui dialogue avec Helm.
 - ✗ Ne possède pas de système de pré-configuration des services.

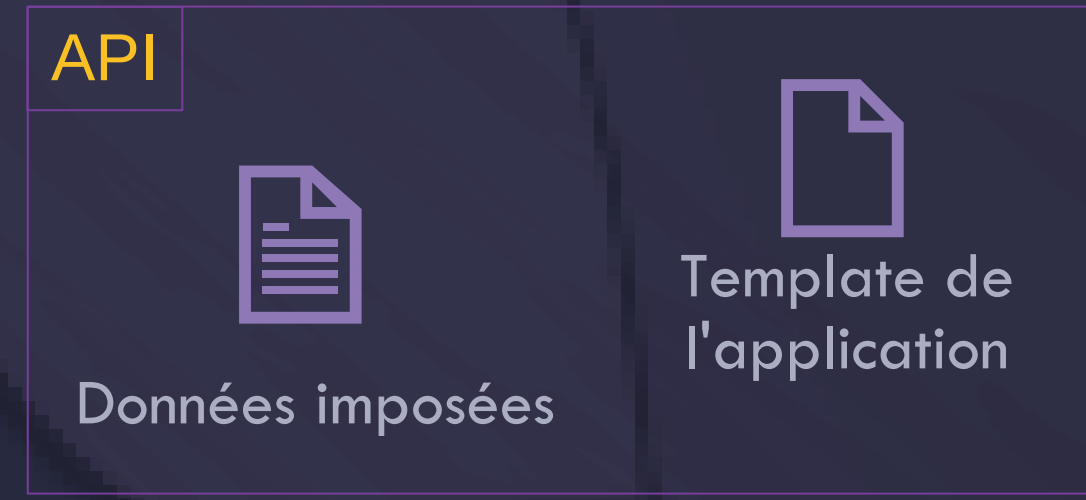
PRÉSENTATION D'ONYXIA

ONYXIA QUÉSAKO ?

- ➔ Un projet open source développé par l'Insee
- ➔ Une passerelle vers plusieurs services
- ➔ Un élément central du DataLab



QUE SE PASSE-T-IL QUAND JE LANCE UN SERVICE ?



CLI Helm



ONYXIA: DU DATAOPS ?

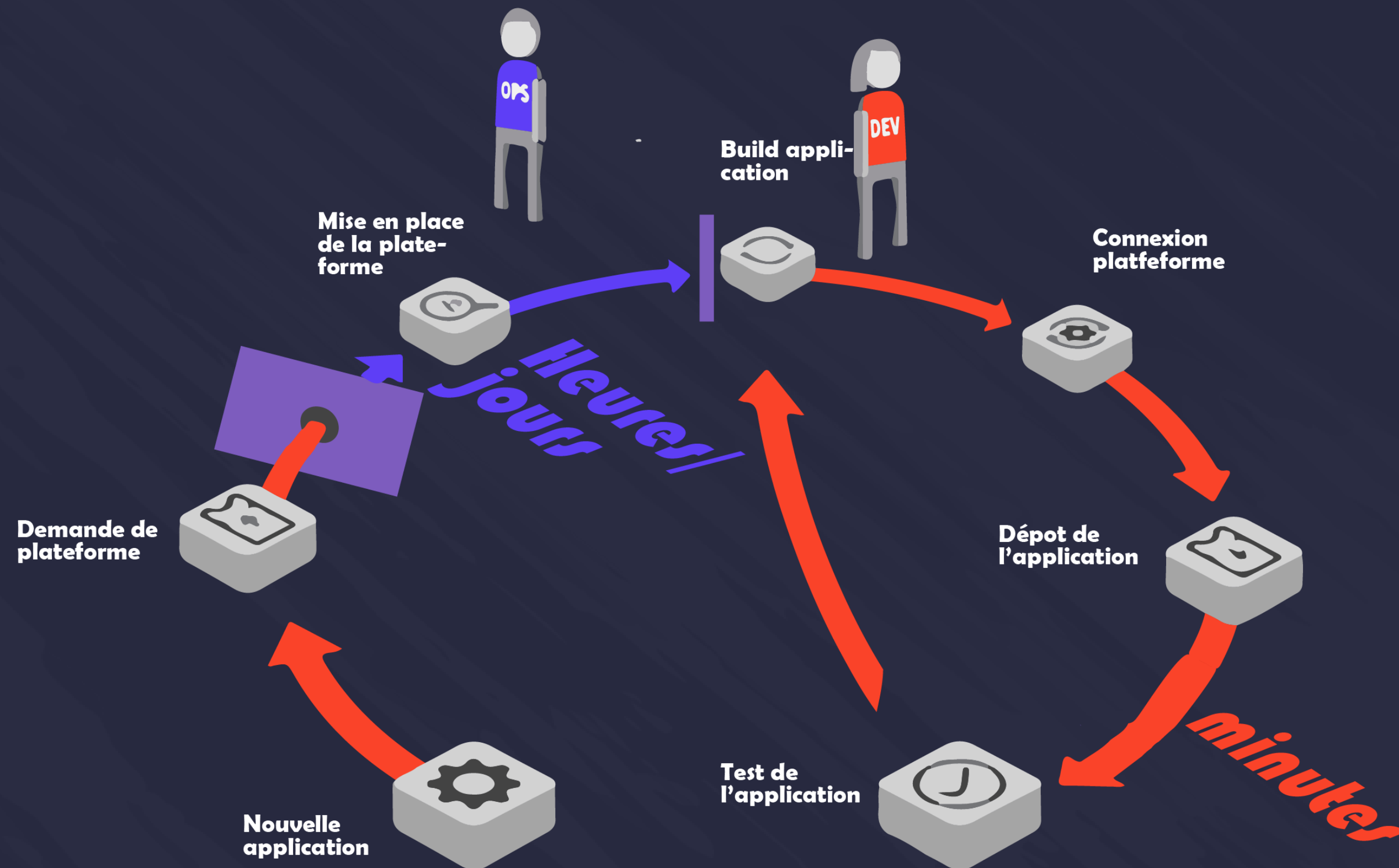
QUELQUES POINTS

- ➔ Des développements compliqués (relativement techniques)
- ➔ Une plateforme désormais compatible Kubernetes
- ➔ Une plateforme toujours compatible Marathon
- ➔ Un fonctionnement équivalent avec les deux orchestrateurs
- ➔ Un projet qui a gagné en notoriété
- ➔ Développement de 2 applications : marathon-to-kub et java-helm-wrapper

OBJECTIF 2 : AMÉLIORER LES PRATIQUES DE CI/CD



POURQUOI DES CONTENEURS EN DEV/QFS ?



LES RAISONS:

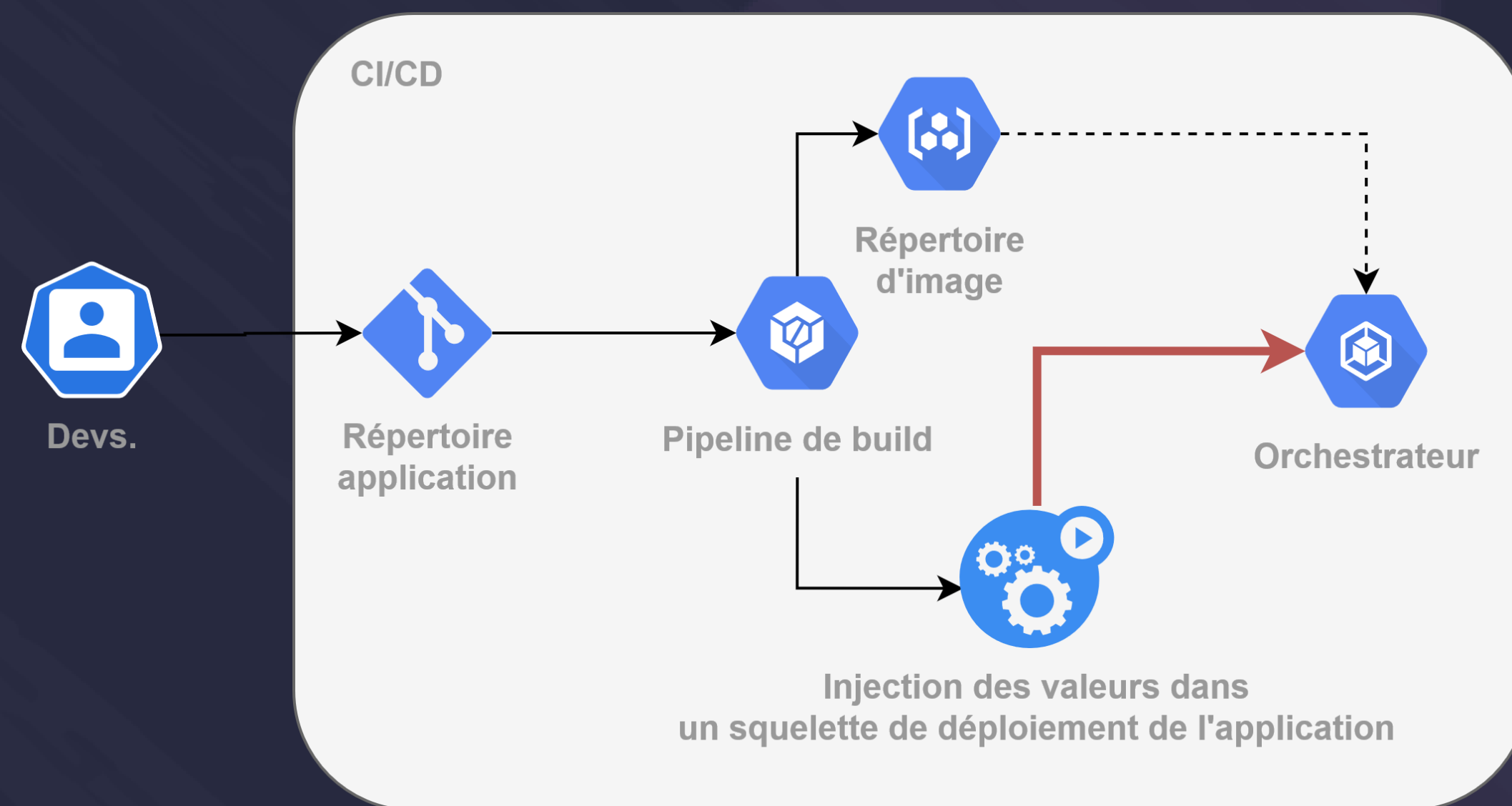
- ➔ Optimiser le déploiement d'une application
- ➔ Apporter de nouvelles pratiques de CI/CD

CI/CD ET CONTENEUR CE QUE L'ON PEUT FAIRE

➡ Automatiser les tâches autour d'une application

➡ Automatiser le déploiement d'un environnement

COMMENT CA MARCHE ?



AVANTAGE :

- ✓ Une infinité d'environnements de tests
- ✓ Un simple envoi de fichier à une API
- ✓ Plus aucune intervention

POUSSER À BOUT L'AUTOMATISATION

AVANCER PAS À PAS

➔ CI/CD

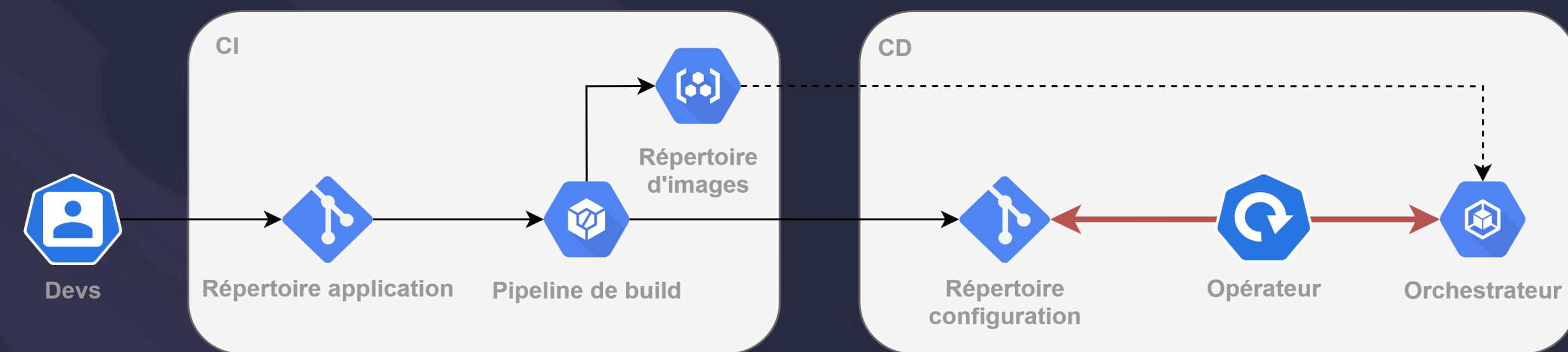
- ✓ Automatisation autour d'une application (Dev/Ops)

➔ Utilisation de Git pour décrire les applications

- ~ Décrire une application as code

➔ GitOps

- ✗ Automatisation du SI as code (ArgoCD utilisé par le ministère de l'intérieur)



ARGO-CD

ARGO C'EST QUOI ?

- ➔ Un projet répondant à la problématique GitOps
- ➔ Un projet open source
- ➔ Un projet uniquement disponible dans l'écosystème Kubernetes



POURQUOI CHOISIR ARGO ?

- | | |
|---|---|
| <ul style="list-style-type: none"> ➔ Tous les avantages du GitOps <ul style="list-style-type: none"> ✓ Une gestion centralisée ✓ Un retour arrière facilité ✓ Une meilleure sécurité | <ul style="list-style-type: none"> ➔ Les avantages d'ARGO <ul style="list-style-type: none"> ✓ Possède une interface graphique et une CLI ✓ Recommandé par la CNCF ✓ Utilisé par le ministère de l'Intérieur |
|---|---|

UNE PLATEFORME CONTENEURISÉE POUR TOUS

CONCLUSION

- ✓ Comprendre et s'appropriier des orchestrateurs de conteneurs
- ✓ Onyxia : modernisation, open source et ajout de fonctionnalités
- ✓ Amélioration des pratiques de CI/CD

RESTE À FAIRE

- ✓ Définir une conduite du changement

ANNEXES



LES GESTIONNAIRES DE PAQUETS

Objectif : faciliter le déploiement d'un service au sein d'un environnement

UNIVERSE

- ➔ Pas vraiment un gestionnaire de paquets, mais un format de templating utilisé par le gestionnaire de paquets DC/OS
- ✓ Utilisable depuis l'interface admin de DC/OS ou depuis la CLI
 - ✗ Seulement 120 paquets sur le répertoire officiel
 - ✗ Non utilisable directement dans Mesos/Marathon

VS

HELM

- ➔ Un vrai gestionnaire de paquets
- ✓ Propose une CLI qui permet d'installer, de mettre à jour, d'arrêter des services (et même plus)
 - ✓ Facile à utiliser
 - ✓ 1200 charts sur le répertoire officiel
 - ✗ Pas d'interface graphique par défaut
 - ✗ Une syntaxe compliquée

UN FONCTIONNEMENT IDENTIQUE

- ✓ Les répertoires contenant les paquets sont exposés directement sur le WEB
- ✓ Les paquets sont composés d'un template de l'application, avec des valeurs par défaut sur-chargeable

LES GESTIONNAIRE DE PAQUETS (COMPLÉMENTS)

Objectif : faciliter le déploiement d'un service au sein d'un environnement

UNIVERSE

➔ Pas vraiment un gestionnaire de paquets, mais un format de templating utilisé par le gestionnaire de paquets DC/OS

- ✓ Utilisable depuis l'interface admin de DC/OS ou depuis la CLI
- ✓ Une description de l'universe exposable directement sur le net
- ✓ 4 fichiers de configuration par paquet :



config.json



marathon.json.mustache



resource.json



package.json

✗ Seulement 120 paquets disponibles sur le répertoire officiel

VS

HELM

➔ Un vrai gestionnaire de paquets

- ✓ Propose une CLI qui permet d'installer, de mettre à jour, d'arrêter des services (et même plus)
- ✓ Facile à utiliser
- ✓ Des paquets pouvant être mis à disposition depuis n'importe quel serveur WEB
- ✓ Pour chaque paquet, 1 dossier et 1 fichier nécessaires :



templates



Values.yaml

- ✓ 1200 charts sur le répertoire officiel
- ! Un vocabulaire spécifique et un format de templating plus compliqué (mais qui apporte plus de fonctionnalités)

DÉPLOYER UNE APPLICATION

	MARATHON	KUBERNETES
FICHIERS/DÉ- PLOIEMENT	1 seul	Plusieurs
LANGAGE	JSON	YAML
ISOLATION	Oui (mais avec configuration)	Oui (namespace)
VAR ENVIRONNE- MENTS	Balise env	Balise env, secret ou configmap
TÉLÉCHARGE- MENT DES RESSOURCES	Balise fetch	Init container
COMMUNIQUER	CLI DC/OS ou appel REST	CLI KUBECTL

Tableau – Points communs et différences **Kubernetes/Marathon** lors du déploiement d'une application

COMPARAISON ARCHITECTURE

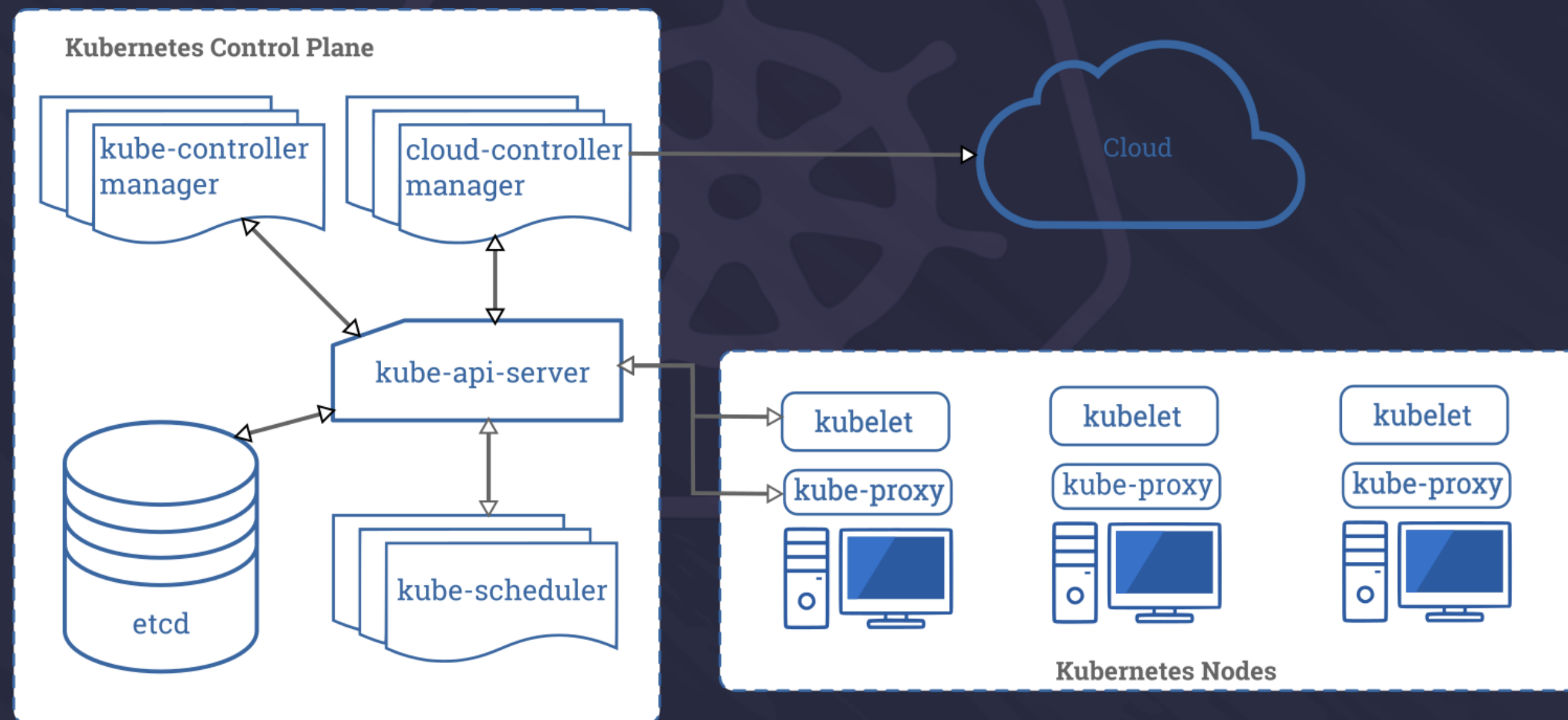


Schéma — Architecture du cluster **Kubernetes**

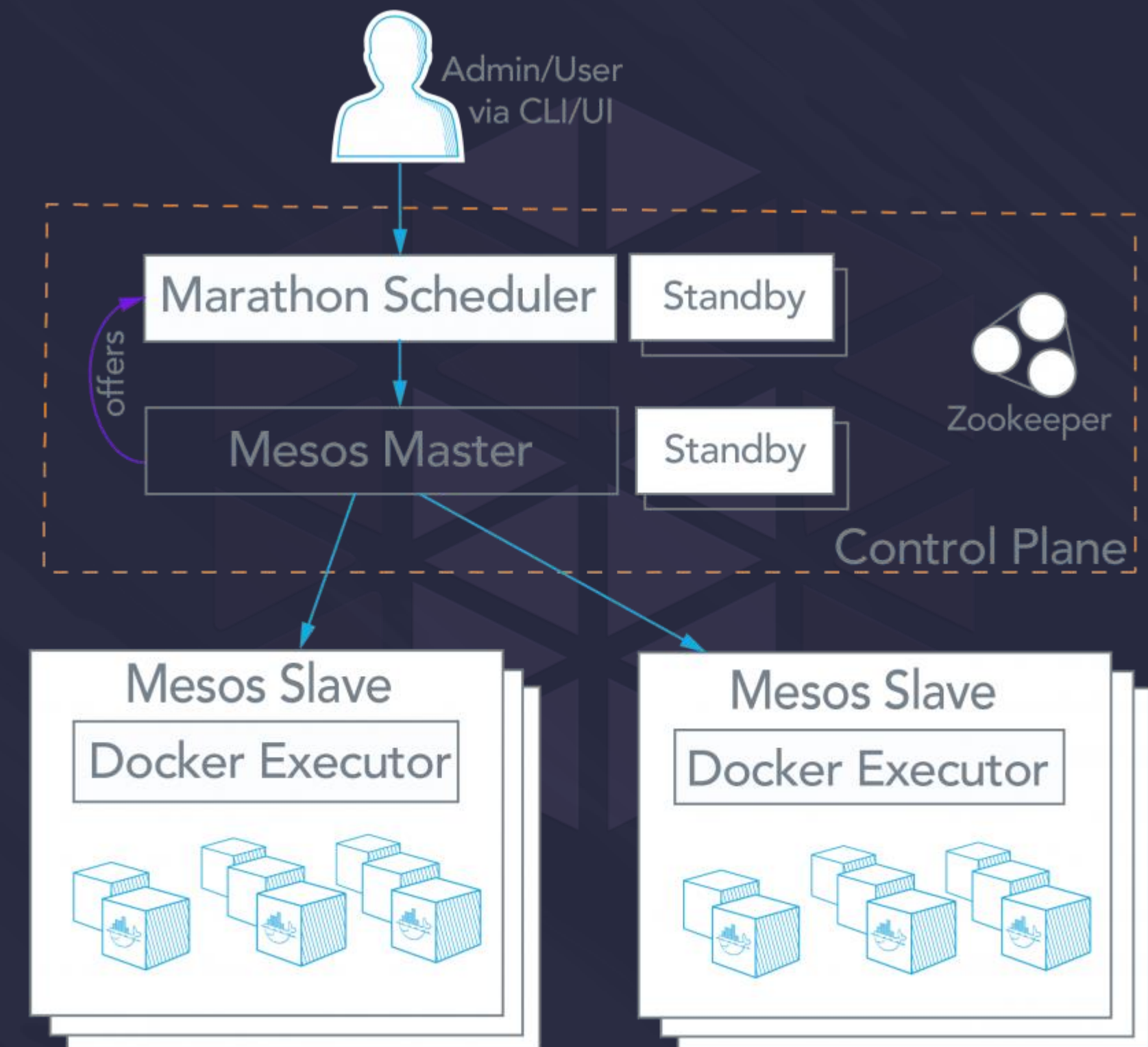


Schéma — Architecture du cluster **Mesos/Marathon**

GIT-OPS

Schéma – Modèle en **Push**

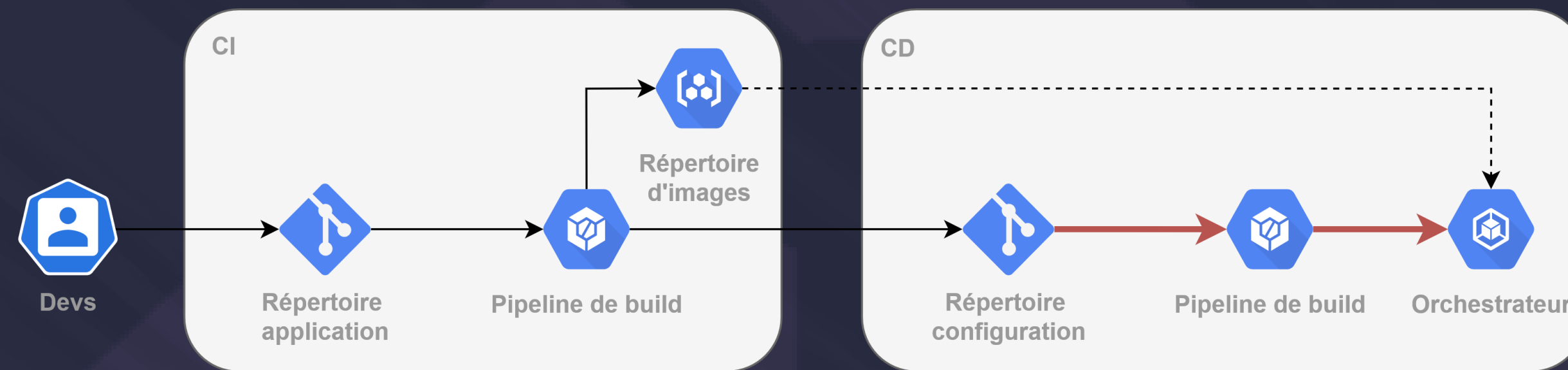
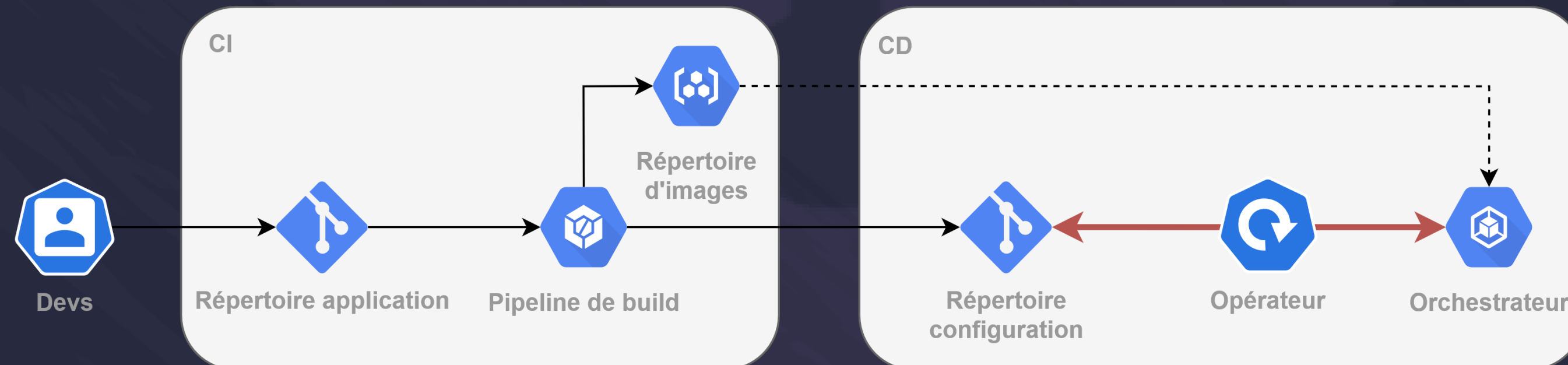


Schéma – Modèle en **Pull**



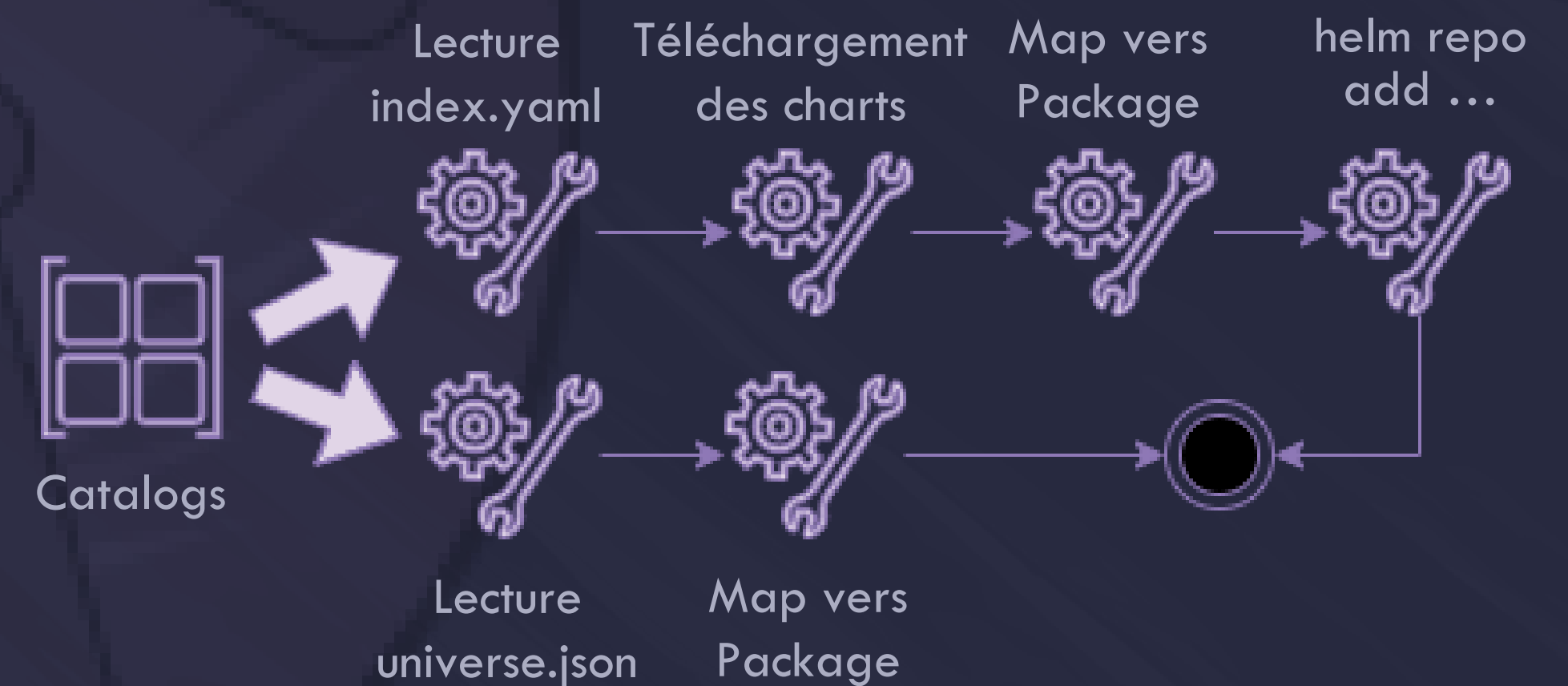
ONYXIA: DE UNIVERSE À HELM

1 - LIRE DES CATALOGUES DE SERVICES

DÉFINIR UN CATALOGUE

- ✓ Choix d'un format de catalogues
- ✓ Repérer les points commun dans la descriptions des paquets (nom, description, version etc...)
- ✗ La configuration par défaut des paquets n'est pas accessible directement dans la description d'un catalogue Helm

CHARGER LES CATALOGUES



CONSTRUIRE UN CATALOGUE

COMMENT DÉFINIR UN CATALOGUE :

- ➔ Un rassemblement de paquets disponibles spécifique à un orchestrateur
- ➔ 2 types de catalogues : Universe (Marathon) et Helm (Kubernetes)

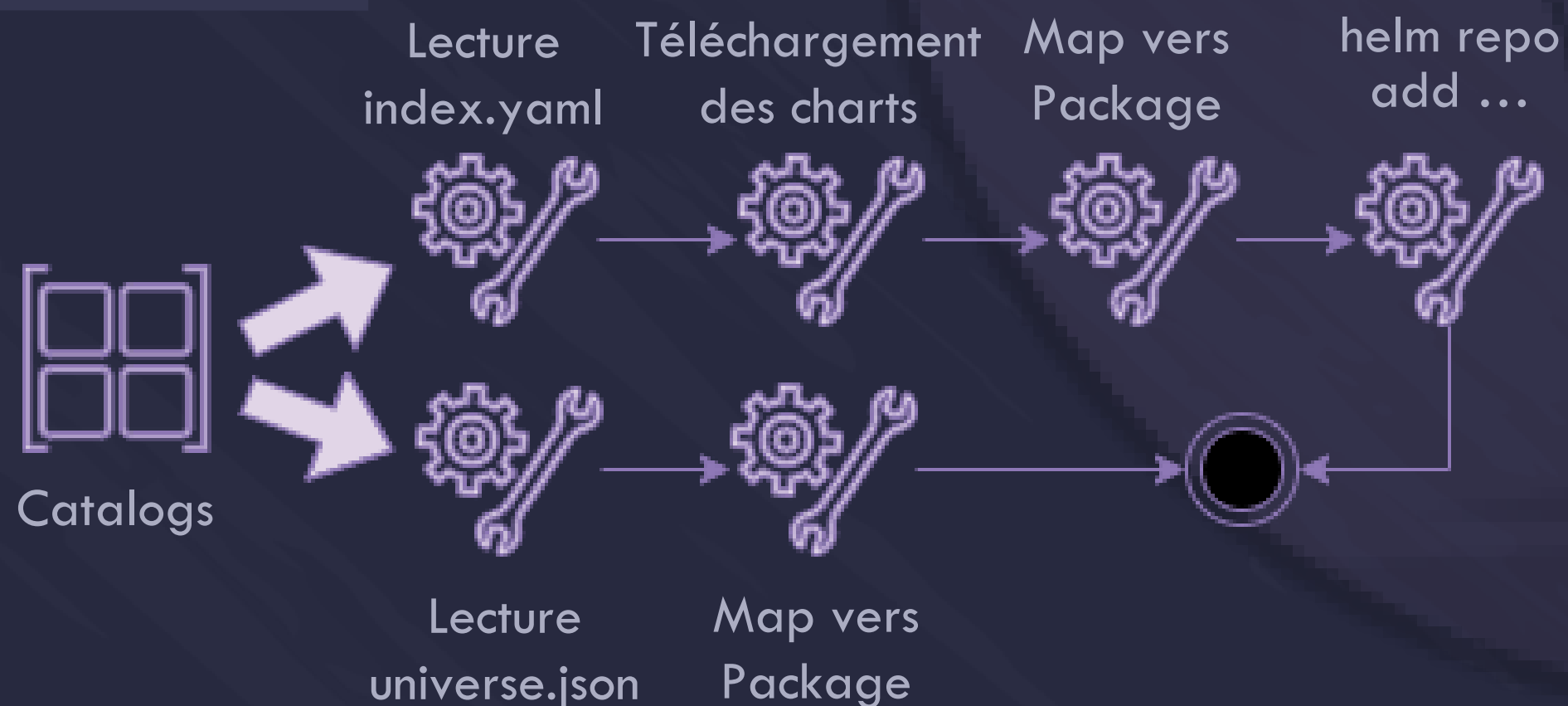
{
Name
Url
Type
+
...

COMMENT DÉFINIR UN PAQUET :

- ➔ Identification des points communs entre Universe Package et Chart.

{
Name
Version
Config
+
...

CHARGEMENT DES CATALOGUES



DIFFICULTÉS RENCONTRÉES

- ✗ Les templates ne sont pas directement accessibles par index.yaml (Helm)
- ✗ Ajout de kubectl et Helm dans l'image d'Onyxia
- ✗ Pas de librairie Helm pour Java

➔ Universe plus simple à manipuler

ONYXIA: DE UNIVERSE À HELM

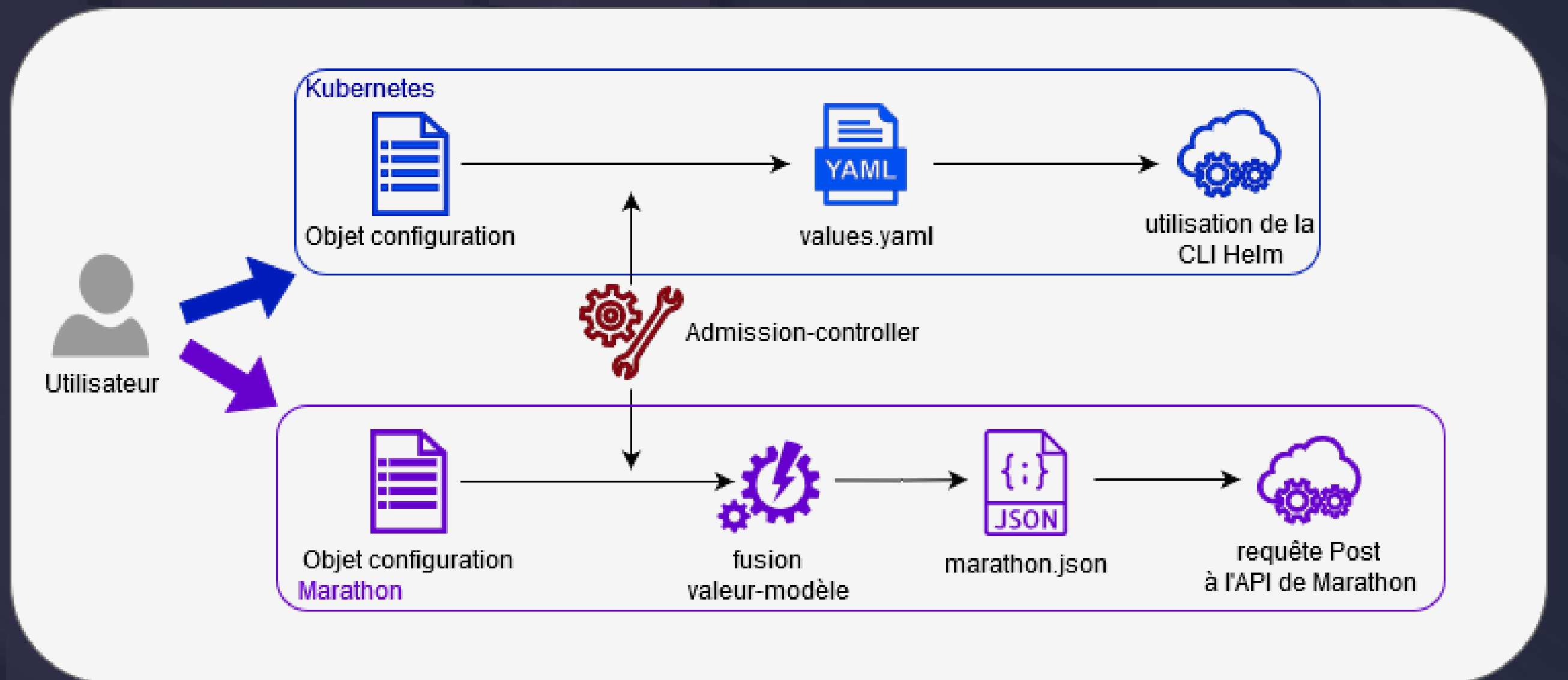
2- LANCER DES SERVICES

ADMISSION CONTROLLER

➔ Contrôler et injecter des valeurs dans le fichier contenant la configuration

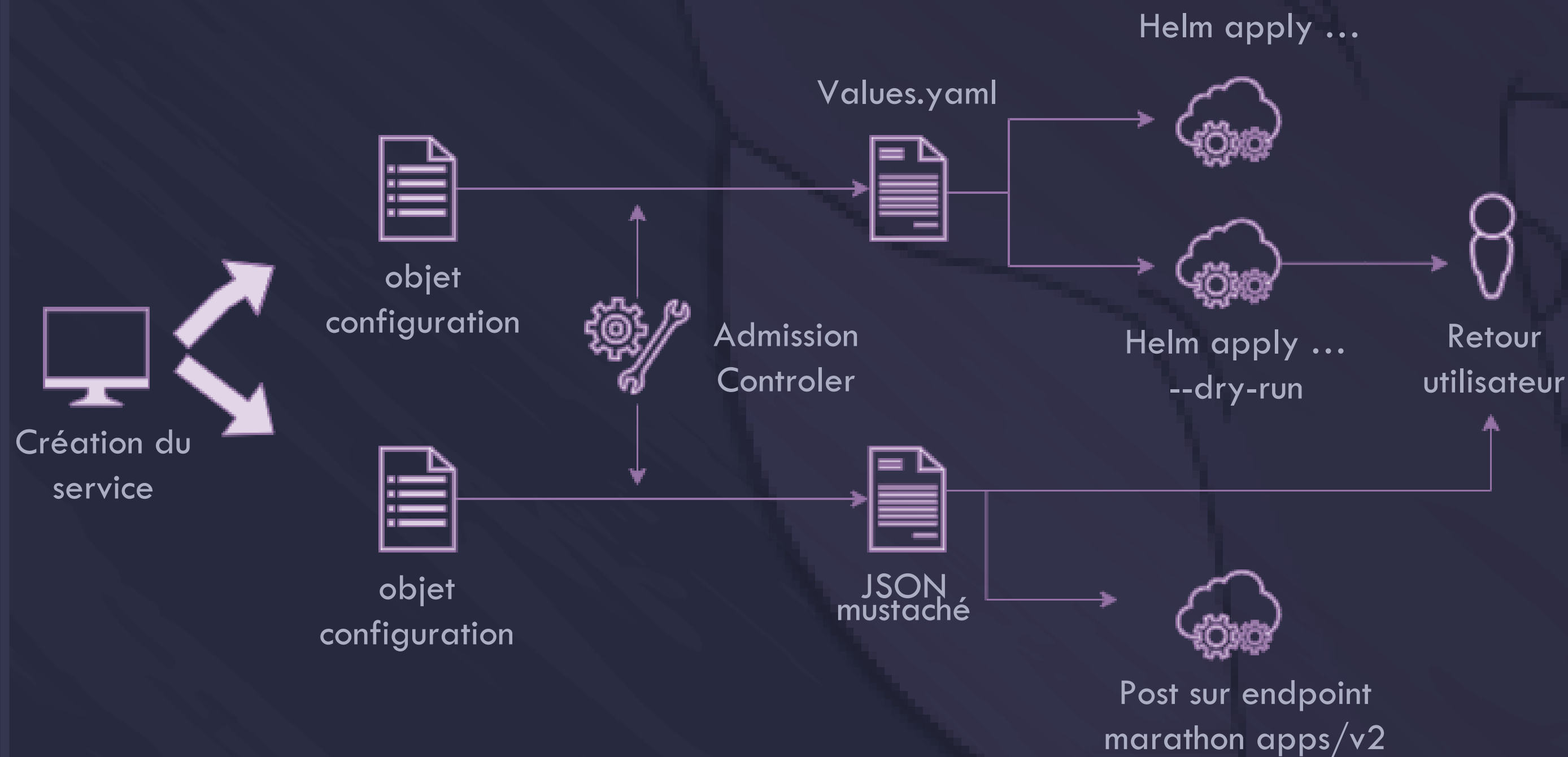
- ✓ Injection des URLs
- ✓ Ajout de labels
- ✗ Nom du service (Universe)
- ✗ Création du namespace (Kubernetes)

WORKFLOW



LANCER UN SERVICE

CRÉATION D'UN SERVICE :



ADMISSION CONTROLLER

➔ Des fonctions qui parcourent l'objet configuration et qui répondent à des besoins

Universe

- ✓ Génération id
- ✓ Gestion URL
- ✓ Gestion Label

Helm

- ✓ Gestion URL

DIFFICULTÉS RENCONTRÉES

Universe

- ✗ Des Admissions controller compliqués à mettre en place
- ✗ Une utilisation détournée de Universe

Helm

- ✗ Création du namespace au préalable
- ✗ Droits admin pour Onyxia dans le cluster Kubernetes

➔ Helm intègre des mécanismes qui facilitent le travail

ONYXIA: DE UNIVERSE À HELM

3 - GÉRER MES SERVICES

➔ Plus facile avec Marathon qu'avec Helm

✗ N+1 Requêtes pour lister les services (Helm)



Impossible de mettre en pause les services (Helm)

GÉRER MES SERVICES

COMPARAISON

	UNIVERSE	HELM
ARRÊTER UN SERVICE	1 requête à marathon	helm rm...
LISTER SERVICE+ OBTENIR DÉTAIL	1 requête permet de lister un dossier	1- helm ls 2- helm get ... pour chaque release

DIFFICULTÉS RENCONTRÉES

- X

Marathon distingue service arrêté et service supprimé mais pas Helm
- X

N+1 requêtes pour lister des services avec Helm

DE NOUVELLES FONCTIONNALITÉS

Choix de la région dans laquelle le service est déployé

1

Intégration d'un terminal qui permet de communiquer directement avec l'orchestrateur

2



LES NOUVEAUTÉS

- ✓ Indépendance de DC/OS
- ✓ De nouvelles fonctionnalités
- ✓ Des technos modernes
- ✓ Une logique open source
- ✓ Une offre mutualisable (communauté Spyrales) et ssp

➔ Une volonté de rester compatible avec les 2 orchestrateurs